GENERATIVE AI WITH GOOGLE EDUGENIE

Google Gemini Powered Learning Assistant

PROJECT REPORT

A complete academic project covering project description, scenarios, workflow, activities, system design, diagrams, implementation, source code, testing, advantages, limitations and conclusion.

Project Title	EduGenie: Google Gemini Powered Learning Assistant
Technology	Generative AI • Google Gemini API • Python • Flask • HTML/CSS/JavaScript • SQLite
Project Type	AI-Based Educational Web Application
Main Users	Students and Educators
Core Purpose	Personalized learning support using Generative AI

Prepared as an academic project document

TABLE OF CONTENTS

1. Project Description
2. Problem Statement
3. Objectives
4. Project Scenarios
5. Project Workflow
6. System Architecture
7. Functional Modules
8. Project Activities
9. Technology Stack
10. Database Design
11. Prompt Design
12. Implementation & Coding
13. Frontend Code
14. Backend Code
15. Gemini Integration
16. Quiz Module
17. Testing
18. Expected Results
19. Advantages & Limitations
20. Future Enhancements
21. Conclusion
22. References

1. Project Description

EduGenie is a Generative AI powered learning assistant designed to support students with personalized explanations, question answering, quiz generation, study notes, revision support and feedback. The system uses the Google Gemini model through an application backend. A student can enter a topic or question, select a learning activity, and receive an AI-generated response in a simple web interface.

Core idea: EduGenie combines a student-friendly interface with a Generative AI service so that learning content can be produced on demand. Instead of providing only fixed textbook content, the application can adapt the explanation to the student's request, level and activity.

Figure 1. EduGenie High-Level Workflow

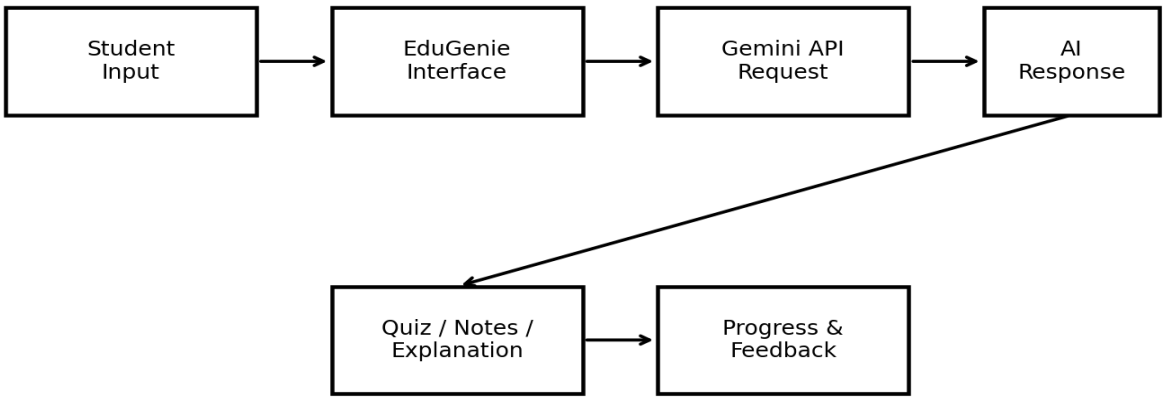


Figure 1 shows the complete high-level flow from student input to AI response, learning activities and progress storage.

2. Problem Statement

Students often need quick explanations, practice questions, summaries and revision material outside classroom hours. Traditional learning applications may contain fixed content and may not adapt easily to different questions. EduGenie addresses this requirement by providing an interactive learning assistant that can generate educational responses based on a student's prompt.

Problem addressed: How can a web-based learning assistant use Generative AI to provide interactive, personalized and activity-oriented academic support while keeping the interface simple?

3. Objectives

- Provide an AI-based question answering interface for students.
- Generate simple explanations, examples and summaries for academic topics.
- Create quizzes and practice questions dynamically.
- Support multiple learning activities such as explain, summarize, quiz and revision.
- Store selected learning history and basic progress information.

• Use prompt templates to produce consistent educational responses.

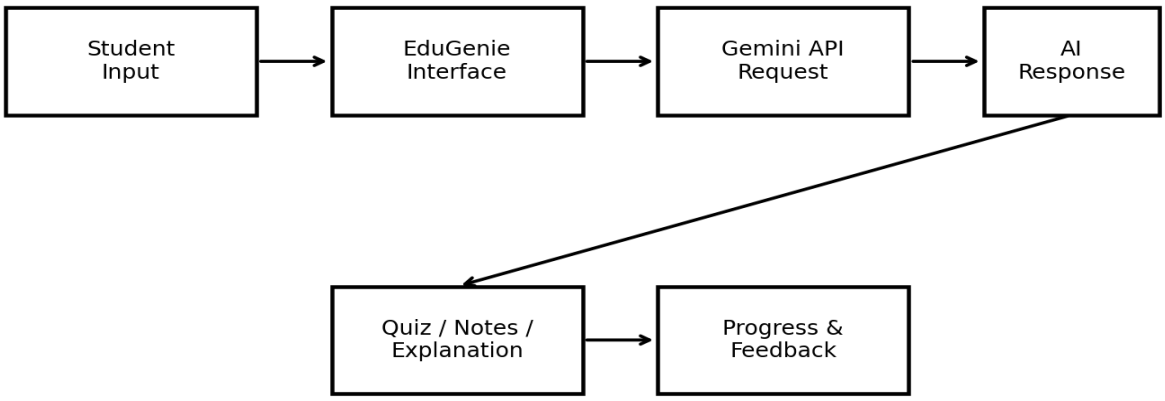
• Provide a modular architecture that can be expanded with additional learning features.

4. Project Scenarios

Scenario	Student Action	EduGenie Response
Topic Explanation	Enters a topic and requests an explanation	Generates a structured explanation with examples
Doubt Solving	Asks a specific question	Provides a step-by-step answer
Quiz Practice	Selects quiz mode and topic	Generates multiple-choice questions and answers
Revision	Requests quick revision	Creates key points and short notes
Exam Preparation	Requests important questions	Generates practice questions based on the topic
Concept Simplification	Requests an easy explanation	Rephrases the concept in simpler language

5. Project Workflow

Figure 1. EduGenie High-Level Workflow



The workflow can be implemented in the following sequence:

1. Student opens the EduGenie web application.
2. Student chooses a learning activity or enters a free-form question.
3. Frontend validates the input and sends it to the backend.
4. Backend builds a structured prompt using the selected activity.
5. Backend sends the prompt to the Gemini API.
6. Gemini generates the response.
7. Backend validates and formats the response.
8. Frontend displays the answer, quiz or notes.
9. Optional history/progress data is stored in SQLite.

6. System Architecture

Figure 2. EduGenie System Architecture

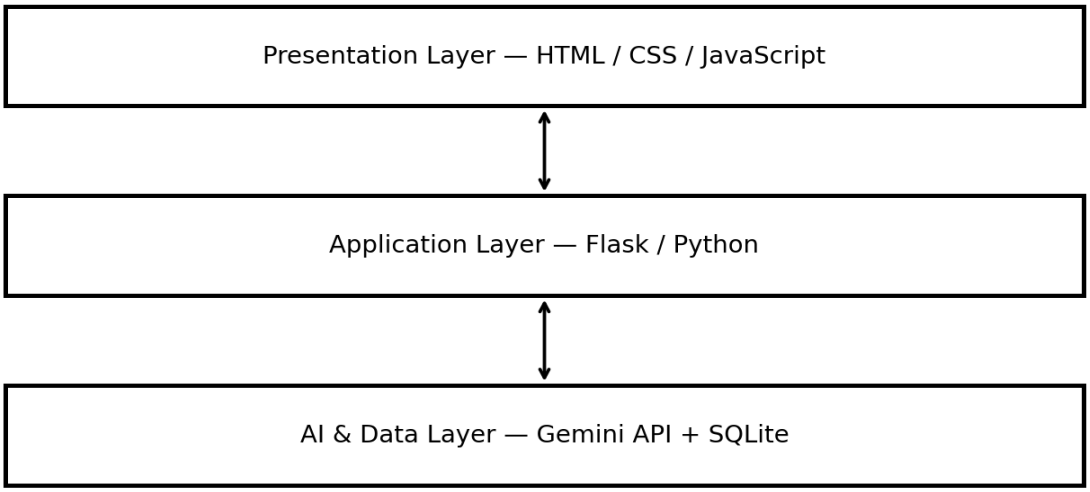
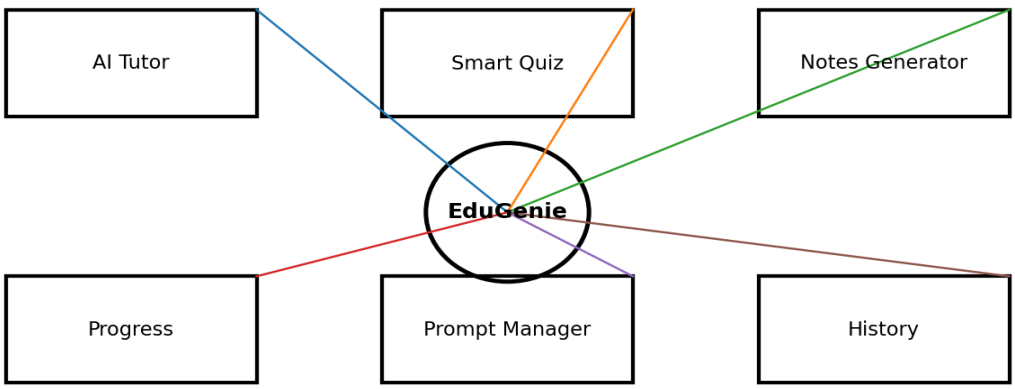


Figure 2. Three-layer architecture: presentation, application and AI/data layers.

The presentation layer manages the user interface. The application layer handles routing, validation, prompt construction and API communication. The AI/data layer contains the Gemini integration and local database used for learning history or progress.

7. Functional Modules

Figure 4. Major Functional Modules



Module	Purpose
AI Tutor	Answers questions and explains concepts.
Smart Quiz	Creates topic-based practice questions.
Notes Generator	Creates concise revision notes.

Prompt Manager	Builds activity-specific prompts.
Progress	Stores simple activity counts and history.
History	Keeps selected previous learning interactions.

8. Project Activities

Activity 1 — Ask EduGenie

Enter a question such as “Explain photosynthesis in simple terms.” The system sends an educational prompt and displays the generated answer.

Activity 2 — Generate Notes

Enter a topic and request key points, definitions and important terms for revision.

Activity 3 — Take a Quiz

Select a topic and number of questions. EduGenie generates MCQs with options and answers.

Activity 4 — Revision Mode

Request a short revision sheet containing key ideas, formulas or definitions.

Activity 5 — Follow-up Question

Use the generated answer as the basis for another question to continue the learning interaction.

9. Technology Stack

Technology	Role
Python	Backend application and Gemini API integration
Flask	Web application framework and routing
Google Gemini API	Generative AI response generation
HTML5	Page structure
CSS3	Interface styling and responsive layout
JavaScript	Client-side interaction and API calls
SQLite	Lightweight local data storage
JSON	Request/response data exchange

10. Database Design

A simple SQLite database can contain two main tables: users and learning_history. For a classroom prototype, authentication can be kept minimal; production deployments should use secure authentication and appropriate privacy controls.

Table	Field	Type	Purpose
users	id	INTEGER	Primary key
users	name	TEXT	Student name
users	created_at	TEXT	Account creation time
learning_history	id	INTEGER	Primary key
learning_history	user_id	INTEGER	Reference to user
learning_history	activity	TEXT	Activity type
learning_history	prompt	TEXT	User request

learning_history	response	TEXT	AI response
learning_history	created_at	TEXT	Timestamp

11. Prompt Design

Prompt engineering is important because the same AI model can produce different outputs depending on instructions. EduGenie uses activity-specific prompt templates. A useful educational prompt should define the role, topic, student level, response format and safety/accuracy expectations.

```
def build_prompt(activity, question, level="college"):
base = f"You are EduGenie, an educational learning assistant. "
base += f"Explain content accurately for a {level} student. "
if activity == "explain":
return base + f"Explain this topic clearly with examples: {question}"
if activity == "summary":
return base + f"Create concise revision notes for: {question}"
if activity == "quiz":
return base + f"Create 5 MCQs on {question}, with 4 options and answers."
if activity == "revision":
return base + f"Create a quick revision sheet for: {question}"
return base + question
```

12. Implementation & Coding

The following reference implementation demonstrates the core backend structure. The API key should be stored in an environment variable and never hard-coded into source files.

requirements.txt

```
Flask
google-genai
python-dotenv
```

13. Frontend Code

```
<!DOCTYPE html>
<html>
<head>
<title>EduGenie</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<main class="container">
<h1>EduGenie</h1>
<p>Google Gemini Powered Learning Assistant</p>

<select id="activity">
<option value="explain">Explain</option>
<option value="summary">Summary</option>
<option value="quiz">Quiz</option>
<option value="revision">Revision</option>
</select>

<textarea id="question" placeholder="Enter your topic or question"></textarea>
<button onclick="askEduGenie()">Generate</button>
<pre id="answer"></pre>
</main>
<script src="/static/app.js"></script>
</body>
</html>
```

Frontend JavaScript

```
async function askEduGenie() {
const question = document.getElementById("question").value.trim();
const activity = document.getElementById("activity").value;
const output = document.getElementById("answer");

if (!question) {
output.textContent = "Please enter a topic or question.";
return;
}

output.textContent = "Generating...";
```

```
const response = await fetch("/ask", {
method: "POST",
headers: {"Content-Type": "application/json"},
body: JSON.stringify({activity, question})
});

const data = await response.json();
output.textContent = data.answer || data.error;
}
```

14. Backend Code

```
from flask import Flask, render_template, request, jsonify
from dotenv import load_dotenv
from google import genai
import os

load_dotenv()

app = Flask(__name__)
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

def build_prompt(activity, question):
role = ("You are EduGenie, a helpful educational assistant. "
"Give accurate, student-friendly answers and use headings where useful.")
if activity == "quiz":
task = f"Create 5 MCQs about {question}. Give 4 options and mark the correct answer."
elif activity == "summary":
task = f"Create concise revision notes about {question}."
elif activity == "revision":
task = f"Create a quick revision sheet about {question}."
else:
task = f"Explain {question} clearly with a simple example."
return role + " " + task

@app.route("/")
def home():
return render_template("index.html")

@app.route("/ask", methods=["POST"])
def ask():
data = request.get_json()
question = data.get("question", "").strip()
activity = data.get("activity", "explain")

if not question:
return jsonify({"error": "Question is required."}), 400

prompt = build_prompt(activity, question)
try:
result = client.models.generate_content(
model="gemini-2.5-flash",
contents=prompt
)
return jsonify({"answer": result.text})
except Exception as exc:
return jsonify({"error": "AI service error. Please try again."}), 500

if __name__ == "__main__":
app.run(debug=True)
```

15. Gemini Integration

The backend creates a Gemini client using an environment variable. The model name should be selected according to the current Google Gemini API documentation and the model available to the project account. API usage, quotas and model availability can change over time.

```
# .env
GEMINI_API_KEY=YOUR_API_KEY_HERE
```

Security rule: Never publish a real API key inside HTML, JavaScript, Git repositories, screenshots or the final academic report.

16. Quiz Module

```
def quiz_prompt(topic, count=5):
return f'''
Create {count} multiple-choice questions for a college student.
Topic: {topic}

For every question provide:
1. Question
2. Four options: A, B, C, D
3. Correct answer
4. One-line explanation

Keep questions educational and avoid ambiguous wording.
'''
```

Example generated quiz structure:

Q1	Which process converts light energy into chemical energy?
A	Photosynthesis
B	Respiration
C	Digestion
D	Transpiration
Answer	A — Photosynthesis

17. Request–Response Sequence

Figure 3. EduGenie Request–Response Sequence

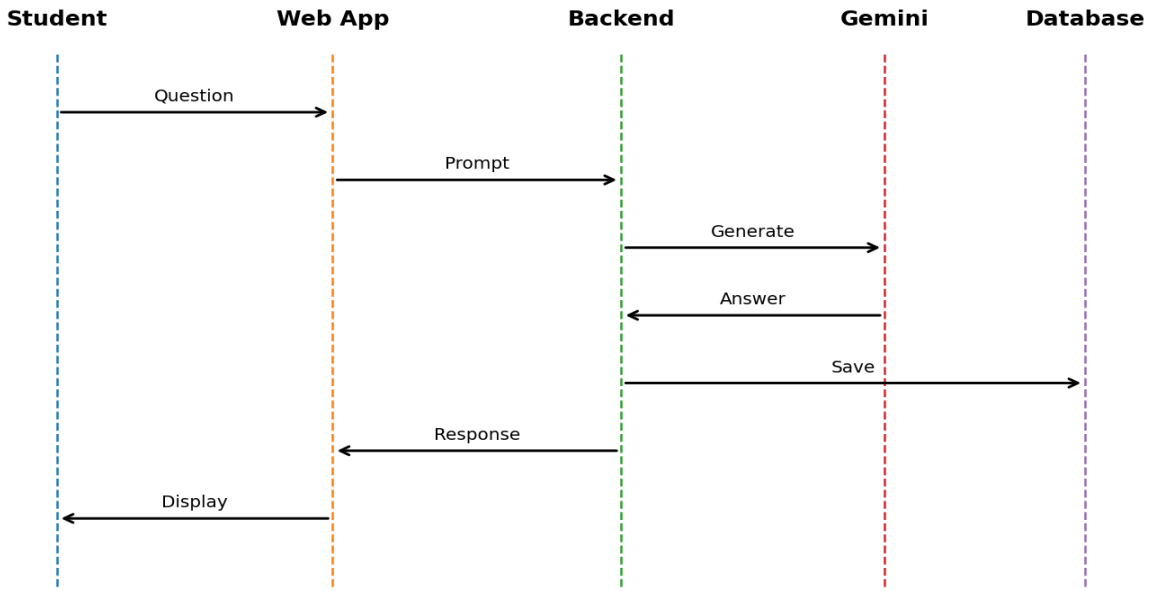


Figure 3. Sequence diagram showing the interaction between the student, web application, backend, Gemini service and database.

18. Project Folder Structure

```
EduGenie/
│
├── app.py
├── requirements.txt
├── .env
├── database.db
│
├── templates/
│   └── index.html
│
└── static/
    ├── style.css
    └── app.js
```

This structure separates backend logic, HTML templates and static frontend files. It can later be expanded into separate modules for authentication, prompts, quiz generation and analytics.

19. Testing

Test Case	Input	Expected Result
TC01	Valid explanation question	AI explanation displayed
TC02	Empty question	Validation message displayed
TC03	Quiz activity + topic	MCQs with options and answers
TC04	Summary activity + topic	Concise notes displayed
TC05	Invalid API response	Friendly error message
TC06	Long input	Backend handles or validates input
TC07	API key missing	Configuration error handled safely

20. Expected Results

• Students can interact with an AI tutor through a simple web interface.

• Educational explanations can be generated on demand.

• Quiz and revision activities can be created dynamically.

• Prompt templates make outputs more consistent across activities.

• The application provides a foundation for tracking learning interactions.

• The modular design allows future expansion into dashboards, voice features and richer educational tools.

21. Advantages and Limitations

Advantages

• Interactive learning support

• On-demand explanations

• Multiple activity modes

• Simple web architecture

• Easy to extend with new prompts and modules

Limitations

• AI responses may contain inaccuracies and should be checked for important academic facts.

• API availability, quotas and model behavior can change.

• A prototype needs stronger authentication, privacy controls and logging safeguards before production deployment.

• The quality of output depends partly on the prompt and supplied context.

22. Future Enhancements

Enhancement	Description
Personalized Learning Path	Recommend topics and activities based on learning history.
Voice Assistant	Allow speech input and spoken explanations.
Teacher Dashboard	Show activity summaries and class-level analytics.
Document Q&A;	Allow students to ask questions about approved course materials.
Multilingual Support	Provide explanations in English, Tamil and other supported languages.
Adaptive Quizzes	Adjust difficulty based on previous answers.
Authentication	Add secure login and role-based access.

23. Conclusion

EduGenie demonstrates how Generative AI can be integrated into an educational web application to create an interactive learning assistant. The project combines a student-friendly interface, a Python/Flask backend, structured prompts, Google Gemini integration and optional database storage. Its core activities—question answering, explanations, summaries, quizzes and revision—provide a practical foundation for AI-assisted learning.

The project is designed as an extensible academic prototype. Future versions can add secure authentication, personalized learning paths, document-based question answering, teacher dashboards, multilingual interaction, voice support and adaptive assessment. Because Generative AI can produce incorrect or incomplete information, important academic content should be verified and the application should include appropriate safety, privacy and quality controls.

Final Outcome: EduGenie provides a clear project model for building a Google Gemini powered learning assistant and demonstrates the complete path from problem definition to system design, coding, testing and future enhancement.

24. References

• Google Gemini API / Google AI documentation — consult the current official documentation for model names, SDK usage, quotas and API changes.

• Python documentation — Python language and standard library reference.

• Flask documentation — web application framework reference.

• SQLite documentation — database reference.

• HTML, CSS and JavaScript standards/documentation for frontend development.

END OF PROJECT REPORT